CS54 · DATA COMMUNICATION & NETWORKING · COMPANION TO THE CIE-1 STRIKE SHEET

GROUND ZERO

Every idea the Strike Sheet assumes you already have — built from nothing, in the order you need it

Read this first if any of these sentences felt like a foreign language

"The transport layer **encapsulates** the message in a **segment** and passes it to the network layer."

"**first = address AND mask**, so the last octet 89 AND 192 gives 64."

"A TCP **socket** is identified by the **4-tuple** and the sender's **cwnd** is halved on 3 duplicate **ACKs**."

Nothing here is extra syllabus. Every page is the underlying idea behind a card in the Strike Sheet, and each lesson ends with a **"this unlocks"** line pointing at the exact questions it lets you answer.

PART A · The mental model

1. What a network actually is
2. The journey of one message
3. Why layers exist — and what a protocol is
4. Encapsulation, and the five object names
5. The four addresses — MAC, IP, port, hostname

PART B · The maths you cannot skip

6. Bits, bytes, octets and powers of 2
7. Decimal ↔ binary ↔ hexadecimal
8. $\log_2$ without a calculator
9. Bitwise AND, OR and NOT
10. Why usable hosts = N − 2

PART C · How the conversation works

11. Processes, clients, servers and sockets
12. Bits vs bytes, Mbps, and the four delays
13. RTT — the unit everything is priced in
14. TCP vs UDP — the fundamental split
15. Sequence numbers, ACKs, timers, windows

HOW TO USE IT

Read **Part A** and **Part C** once, straight through — about 35 minutes. **Part B** is reference: work the three conversions on paper, then come back to it whenever a Unit 3 numerical stalls. The last two pages are a **glossary** and an **acronym decoder** — use them while reading the Strike Sheet, not before.

THE HONEST SHORTCUT

If you have under an hour: read **Lesson 4** (encapsulation), **Lesson 5** (the four addresses), **Lesson 9** (bitwise AND/OR) and **Lesson 15** (sequence numbers and ACKs). Those four carry about 80 % of the vocabulary in the Strike Sheet.

Companion to **DCN_CIE1_Strike_Sheet.pdf** · Units 1, 2 and 3 up to IPv4 addressing · Same colour code: **blue Unit 1**, **violet Unit 2**, **teal Unit 3**, **amber memory hook**, **pink trap**, **green analogy**.

PART A The mental model

Lessons 1 – 5 · read straight through

01 What a network actually is

A network is just devices joined by links so they can exchange data. That's all.

Formally: **a network is the interconnection of a set of devices capable of communication.** Everything in this course is a detail of *how* that exchange is organised.

Two kinds of device — this distinction matters all term

Type	What it is	Examples
Host (end system)	A device that **produces or consumes** the data. The message starts or ends here.	Laptop, desktop, phone, server, security system
Connecting device	A device that only **moves data along**. It never creates the message.	**Router** (joins networks), **switch** (joins devices), **modem** (changes the form of data), hub, repeater

A **node** is any device that can send and/or receive data — it covers both kinds.

Why you care: the exam repeatedly asks *"how many layers is each device involved in?"* The answer depends entirely on this split — a host runs all 5 layers, a router only 3, a switch only 2. A connecting device doesn't need the top layers because it never reads the message, only the envelope.

The four connecting devices, by layer

Device	Layer	What it does
Repeater	1 Physical	Amplifies / regenerates a weak signal to extend range. Understands **nothing** — just boosts.
Hub	1 Physical	Connects devices and **broadcasts to all ports**, no filtering. Dumb.
Switch	2 Data link	Forwards only to the correct device using the **MAC address**.
Router	3 Network	Routes between **different networks** (LAN ↔ Internet) using the **IP address**.

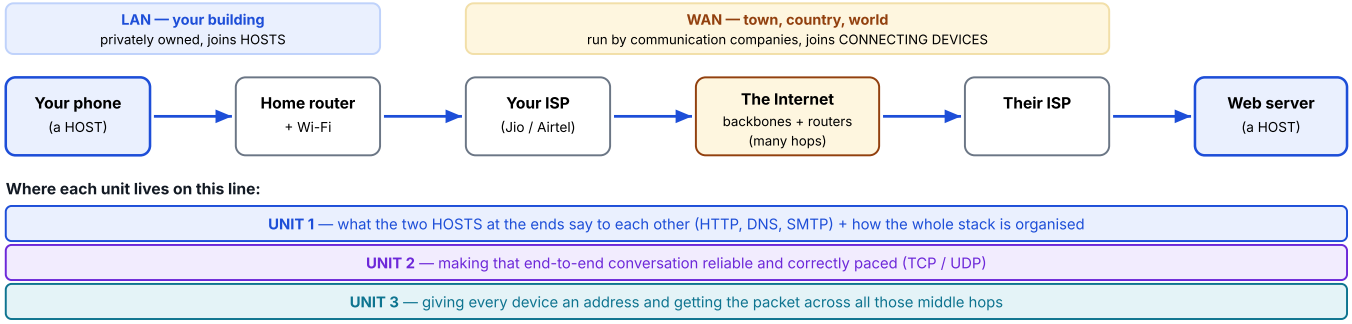
ANALOGY A **hub** shouts your message into a crowded room and hopes the right person hears. A **switch** walks it to the right desk in the office. A **router** takes it to a different building.

HOOK The higher the layer, the smarter the device. **Repeater 1, Hub 1, Switch 2, Router 3.**

THIS UNLOCKS Strike Sheet Q8 (host 5 / router 3 / switch 2) and Q10 (topologies, hub failure).

02 The journey of one message — the picture to hold in your head

Everything in Units 1, 2 and 3 is one slice of this single journey. When a question confuses you, locate it on this line first.



The words on that picture

Term	Meaning
LAN	Local Area Network. **Privately owned**, one office / building / campus. **Joins hosts.**
WAN	Wide Area Network. Town, state, country, world. Run by communication companies and **leased**. **Joins connecting devices.**
internet small i	**Any** two or more networks joined together.
Internet capital I	The one global internet, made of thousands of networks. Three levels: **backbones → provider networks → customer networks**.
ISP	Internet Service Provider. Backbones are **international** ISPs; provider networks are **national/regional** ISPs.
Peering point	The switching system where two backbones connect to each other.
Hop	One step of the journey. **A hop is a host or a router.**

The one sentence that makes layering click

Look at the picture again. The **message** ("GET me this web page") only matters at the **two ends**. Every box in the middle doesn't care what the message says — it only needs to know **where to send it next**.

That is *exactly* why the stack is split into layers. The top layers are the **conversation**; the bottom layers are the **delivery**. Middle boxes implement only the delivery layers, so they stay cheap and fast.

ANALOGY You post a letter. The **letter** is for your friend. The **postal workers** never open it — they only read the envelope, and each one only needs to know the *next* sorting office, not the whole route. The letter is the application layer; the envelope is everything below.

HOOK **Ends talk, middles forward.** Application / transport / network are **end-to-end**. Data link / physical are **hop-to-hop**.

THIS UNLOCKS Strike Sheet Q8 (end-to-end vs hop-to-hop), Q10 and the whole Unit 1 Section A vocabulary.

03 What a protocol is, and why anyone bothered with layers

A protocol is a set of rules that the sender, the receiver and every device in between agree to follow.

Not code, not hardware — an **agreement**. "Port 80 means web traffic." "A line containing only a full stop ends an email." "Reply `200 OK` if you found the file." Both sides must already know the rules, or the bytes are meaningless.

The textbook story — Maria and Ann (worth knowing, it's examinable)

Scenario 1 — face to face. Two neighbours talk in the same language. Communication happens in **one layer**. Their protocol has 5 rules: greet on meeting; keep vocabulary to the level of the friendship; don't speak while the other speaks; it's a dialog not a monolog; exchange nice words on leaving.

Scenario 2 — by post, encrypted. Ann moves away. They now write letters, and encrypt them so an intercepted letter can't be read. Communication now happens in **three layers**, each with its own machine:

Layer	Maria (sender)	Ann (receiver)
3	Talks to the machine as if it were Ann (plaintext)	Machine speaks the plaintext to Ann
2	Encrypts plaintext → ciphertext	Decrypts ciphertext → plaintext
1	Puts ciphertext in an envelope, adds the addresses, mails it	Takes the letter out of the mailbox and the envelope

Modularity is the point. If a better encryption comes along they replace **only the layer-2 machine**. A layer is a **black box** — you care about its inputs and outputs, never how it works inside. Two machines giving the same output for the same input can replace each other, even from different manufacturers.

The two principles of protocol layering

1. For **bidirectional** communication, each layer must perform **two opposite tasks**, one in each direction (encrypt / decrypt, send / receive).
2. The two objects **under each layer at both sites must be identical** (the ciphertext under layer 2 is the same on both sides).

Advantages and the one disadvantage

Advantages: it **separates the service from the implementation**; and **intermediate devices don't need all the layers** — without layering they would be as complex as end systems, making everything expensive.

Disadvantage: some argue a **single layer** would make the job simpler.

Logical connection — a phrase you must be able to define

Layer-to-layer communication is **logical**, meaning **imaginary**. Maria's layer-2 machine behaves *as if* it hands the ciphertext straight to Ann's layer-2 machine. In reality the data goes all the way down, across, and all the way up.

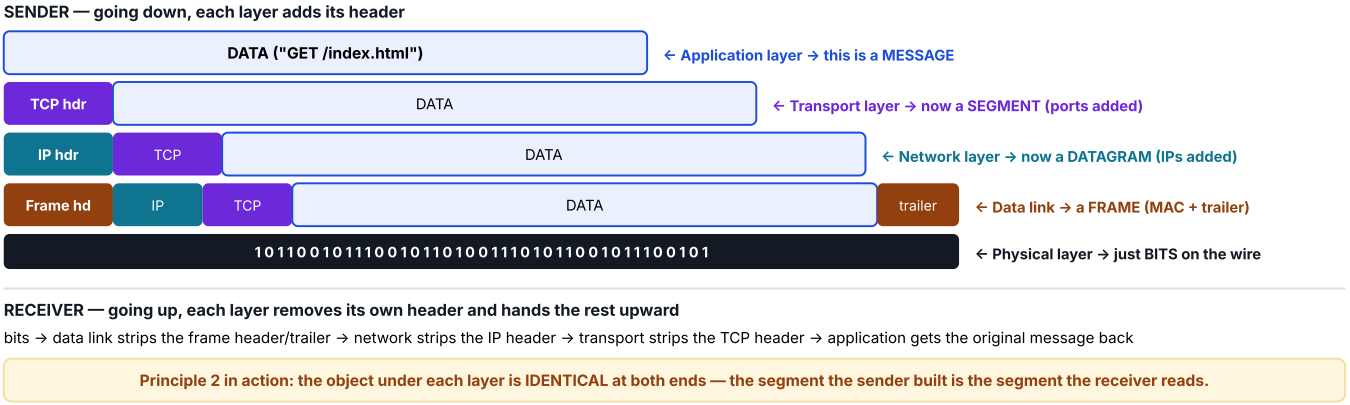
TRAP **Only the physical layer really moves anything.** Every other "connection" is a convenient fiction. When a question says *"explain the logical connections"*, that sentence is worth a mark on its own.

THIS UNLOCKS Strike Sheet Q8 (logical connections) and the protocol-layering theory in Unit 1 Section A.

04 Encapsulation — and why the same data has five different names

Going down the stack, each layer wraps what it was given in its own header. Going up, each layer strips its own header off.

That's it. **Encapsulation** = adding headers on the way down. **Decapsulation** = removing them on the way up. The data doesn't change — it just collects and sheds envelopes. **The five "object" names in the syllabus are simply the same data at five stages of wrapping.**



The five names — learn this table cold

Layer	Object is called	What its header adds
Application	**Message**	nothing yet — this is the user's data

ANALOGY You write a note (**message**). You put it in an envelope marked "for the accounts *department*" (**segment** — port). You put that in a bigger envelope with the *company's* street address (**datagram** — IP). The courier puts that in his bag labelled with the *next depot* (**frame** — MAC),

Transport	**Segment** (TCP) or **user datagram** (UDP)	**port numbers** → which *program*
Network	**Datagram** / packet	**IP addresses** → which *computer*
Data link	**Frame**	**MAC addresses** → which *next device* + error check
Physical	**Bits**	nothing — it's electricity, light or radio now

HOOK **M**y **S**ister **D**raws **F**unny **B**irds = Message, Segment, Datagram, Frame, Bits.

and the van drives (**bits**). At the far end everything is unwrapped in reverse and the note is read.

The two facts people get wrong

1. The MAC address changes at every hop; the IP address does not. The outer bag is re-labelled at each depot, but the company's street address stays on the inner envelope the whole way. This single fact explains why there are two different addresses at all.

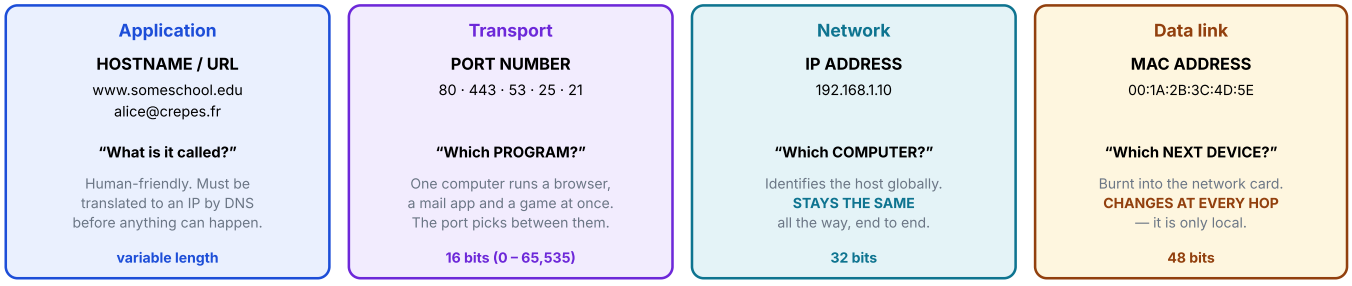
2. Headers are overhead. A TCP header is 20 bytes and an IP header is 20 bytes, every single packet. That's why **MSS = 1460 = MTU 1500 − 20 − 20**, and why UDP's **8-byte** header is considered a real advantage over TCP's 20.

TRAP "Packet" is used loosely in speech for *anything* being sent. In an exam answer, use the **exact** name for the layer you're discussing — segment at transport, datagram at network, frame at data link.

THIS UNLOCKS Strike Sheet Q8, Q9 (encapsulation figure), Q16 (TCP header), Q19 (UDP header) — and every sentence in the PDF containing the word "segment".

05 The four addresses — and why one is not enough

Each layer has its own kind of address, because each layer answers a different question.



THE ANALOGY THAT FIXES THIS FOREVER Sending a parcel to a friend in a large apartment block:
Hostname = "Rahul's place" — what you call it. Useless to the courier until you look up the actual address.
IP address = the building's street address: *12 MG Road, Bengaluru*. Written on the parcel once, never changed.
Port number = the flat number: *#402*. Gets it to the right *person inside* the building.
MAC address = the label on the delivery van's bag saying which *depot it goes to next*. Re-labelled at every depot along the way.

Why this is the single most useful lesson here

Half of the Strike Sheet's sentences are about one of these four addresses:

- **DNS** (7 past questions) exists only to turn a **hostname into an IP address**.
- **Multiplexing / demultiplexing** (4 questions) is entirely about **choosing the right port**.
- **The whole of Unit 3** (14 questions) is about structuring the **IP address**.
- **Switches vs routers** is MAC vs IP.

Ports — the numbers to know

Port	Service	Transport
20 / 21	FTP data / FTP control	TCP
25	SMTP (sending mail)	TCP
53	DNS	**UDP**
80	HTTP	TCP
110	POP3 (reading mail)	TCP
143	IMAP (reading mail)	TCP
443	HTTPS	TCP

Ports **0–1023** are **well-known** ports, reserved for standard services. A client program gets a random high port (like **19157** or **26145**) assigned by its operating system — that's why past-paper examples use odd-looking numbers.

HOOK A **socket** is just **IP + port** glued together: *which computer + which program on it*. When the Strike Sheet says "UDP socket = 2-tuple, TCP socket = 4-tuple", it means UDP identifies a conversation by the **destination** pair only, while TCP needs **both** ends' pairs.

TRAP An IP address identifies a **host**, not a **process**. That one sentence is the entire reason the transport layer exists — the network layer gets data to the right machine, the transport layer gets it to the right program. Examiners love the phrase **"process-to-process delivery"**.

THIS UNLOCKS Strike Sheet Q1–Q3 (DNS), Q9 (addresses per layer), Q18 (mux/demux, sockets, tuples) and all of Unit 3.

05b Quick reference — the OSI and TCP/IP stacks side by side

Two models for the same thing. **OSI has 7 layers and is a teaching model. TCP/IP has 5 and is what actually runs the Internet.** The syllabus wants both, and the difference is always the same question.

OSI (7)	TCP/IP (5)	Job in one phrase
7 Application	**Application**	What the user wants
6 Presentation		Format, encrypt, compress
5 Session		Manage the conversation
4 Transport	Transport	Right program, reliably
3 Network	Network	Right computer, across networks
2 Data link	Data link	Right next device, this link
1 Physical	Physical	Bits on the medium

Session and presentation are missing from TCP/IP. Two reasons: (1) TCP/IP has more than one transport protocol and some session functions already live inside them; (2) the application layer isn't one piece of software, so an app that needs session or presentation features **builds them in itself**.

Why OSI never took over — 3 reasons: it was finished **too late** (TCP/IP was already deployed with time and money invested), some layers were **never fully defined**, and it had **poor performance** when implemented.

HOOK **ISO is the organisation. OSI is the model.** ISO was founded in 1947; OSI arrived in the late 1970s. Mixing these two up is an easy mark to lose.

ANALOGY OSI is the **architect's drawing** with every room labelled. TCP/IP is the **building that actually got built** — a few rooms got merged on site.

THIS UNLOCKS Strike Sheet Q8 and Q9.

PART B The maths you cannot skip

Lessons 6 – 10 · Unit 3 is 80 % numerical

06 Bits, bytes, octets — and the powers of 2 you must know by sight

A bit is one 0 or 1. Eight bits make one byte. In networking a byte is called an octet.

An **IPv4 address is 32 bits** = 4 octets, written as four decimal numbers with dots: `192.168.1.10`. Each octet is 8 bits, so each can hold **0 to 255** (that is 2^8 = 256 different values, counting from 0).

The **address space** is the total number of addresses a protocol can express. With *b* bits it is $\mathbf{2^b}$. For IPv4: $\mathbf{2^{32}}$ **= 4,294,967,296** — about 4.3 billion. That number is the reason the entire second half of Unit 3 exists.

Why 2^n and nothing else

Every time you add one bit you **double** the possibilities. One bit = 2 values. Two bits = 4. Three bits = 8. This is why **every block size in Unit 3 is a power of 2** and why a request for 1000 addresses gets you 1024 — there is no such thing as a 1000-address block.

TRAP Count from **zero**. 8 bits give **256** values but the **largest** is **255**. An octet of 301 is invalid; so is a leading zero like `045`.

The table to memorise — seriously, memorise it

n	0	1	2	3	4	5	6	7	8
2^n	1	2	4	8	**16**	32	**64**	**128**	**256**
n	**9**	**10**	**11**	**12**	**13**	**14**	**16**	**21**	**24**
2^n	512	**1024**	2048	4096	8192	16,384	**65,536**	2,097,152	**16,777,216**

Those bold ones are the numbers that appear in actual exam questions: **16, 64, 128, 256** (subnet sizes), **1024** (the ISP example), **65,536** (class B), **16,777,216** (class A), **2,097,152** (number of class C blocks).

HOOK You only need to count doubles from 1: **1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024**. Everything bigger is 1024 doubled a few more times. And $\mathbf{2^{10} \approx 1000}$, so $2^{20} \approx$ a million and $2^{30} \approx$ a billion — that's how you sanity-check $2^{32} \approx$ 4 billion in two seconds.

THIS UNLOCKS Every Unit 3 numerical — Strike Sheet Q20–Q25.

07 Decimal ↔ binary ↔ hexadecimal — the 30-second methods

You will have to convert one octet to binary in the exam. Here is the only method you need.

Write the place values once, then just tick them off. Example: convert 89 to binary.

	128	64	32	16	8	4	2	1	
value									
bit	0	1	0	1	1	0	0	1	= 01011001

Working, left to right: 128 > 89, so **0**. 64 ≤ 89, so **1**, remainder 89−64 = 25. 32 > 25, so **0**. 16 ≤ 25, so **1**, remainder 9. 8 ≤ 9, so **1**, remainder 1. 4 > 1, so **0**. 2 > 1, so **0**. 1 ≤ 1, so **1**, remainder 0. Check: 64 + 16 + 8 + 1 = 89. ✓

Going back is even easier: add up the place values wherever there is a 1.

Binary → decimal

Add the place values under the 1s.

```
  128 64 32 16  8  4  2  1
    1  1  0  0  0  0  0  0
= 128 + 64
= 192

  128 64 32 16  8  4  2  1
    1  1  1  0  0  0  0  0
= 128 + 64 + 32
= 224
```

This is exactly how you turn a **/26** into **255.255.255.192**: the prefix leaves 2 ones in the last octet, and 128 + 64 = 192.

Decimal → hexadecimal

Hex digits are **0–9 then A B C D E F** (A = 10 … F = 15). Each octet becomes exactly **2 hex digits**: split the 8 bits into two groups of 4, convert each.

bits	0000	0001	0010	0011	0100	0101	0110	0111
hex	0	1	2	3	4	5	6	7
bits	1000	1001	1010	1011	1100	1101	1110	1111
hex	8	9	A	B	C	D	E	F

The slide's worked example

```
128.11.3.31

128 = 1000 0000 = 80
 11 = 0000 1011 = 0B
  3 = 0000 0011 = 03
 31 = 0001 1111 = 1F

→ 800B031F
```

And the "spot the error" exercise:

111.56.**045**.78	leading zero
221.34.7.8.**20**	5 octets
75.45.**301**.14	> 255
11100010.23.14.67	mixed notations

HOOK The only three notations are **binary, dotted-decimal and hexadecimal**. Never mix them in one address.

THIS UNLOCKS Strike Sheet Q20 (mask in binary), Q27 (notations) and every "first/last address" calculation.

08 $\log_2$ — what it means and how to do it in your head

$\log_2$ N simply asks: *"2 to the power of what gives N?"* Nothing more.

$\log_2 256$ = **8**, because $2^8 = 256$. $\log_2 64$ = **6**. $\log_2 16$ = **4**. You are **reading your powers-of-2 table backwards** — that is the entire operation. No calculator, no log tables.

It appears in exactly one formula in your syllabus:

$$n_{sub} = 32 - \log_2 N_{sub}$$

Which in plain words is: **"the prefix length is 32 minus the number of host bits, and the number of host bits is however many doublings it takes to reach the block size."**

ANALOGY If 2^n is "fold the paper n times", then $\log_2$ is "how many folds did that take?" Doubling and halving are the same question asked in two directions.

The lookup you will actually use

Block size N	4	8	16	32	64	128	256	512	1024
$\log_2$ N (host bits)	2	3	4	5	6	7	8	9	10
prefix = 32 − that	/30	/29	**/28**	/27	**/26**	**/25**	/24	**/23**	/22

Read it right to left and you have the other direction too: a **/26** has 32 − 26 = **6 host bits**, so 2^6 **= 64 addresses**.

HOOK **Bigger prefix = smaller block.** /24 is 256 addresses; /26 is 64; /28 is 16. The slash counts the **network** bits, so the more bits the network takes, the fewer are left for hosts.

TRAP If a requirement is **not** a power of 2, **round up**. 1000 addresses → 1024 → /22. 50 → 64 → /26. 120 → 128 → /25. You can never allocate exactly 1000, because n would not be a whole number.

THIS UNLOCKS Strike Sheet Q21, Q22, Q23 (all subnetting design).

09 Bitwise AND, OR and NOT — the operation behind every addressing sum

"Bitwise" means: line the two numbers up in binary and apply the rule to each column independently. No carrying.

The three truth tables

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	**1**

AND = both must be 1. Think of it as a filter that **keeps** a bit only where the mask has a 1, and **zeroes** everything else.

A	B	A OR B
0	0	0
0	1	**1**
1	0	**1**
1	1	**1**

OR = at least one is 1. It **forces** a bit to 1 wherever the other operand has a 1.

A	NOT A
0	**1**
1	0

NOT = flip every bit. So NOT(255.255.255.192) = 0.0.0.63.

Why Unit 3 uses them — worked on 156.180.150.89/26

```
The mask for /26: 26 ones, then 6 zeros
mask = 11111111.11111111.11111111.11000000
     = 255.255.255.192

FIRST address = address AND mask   → zero the host bits
   last octet 89 = 0 1 0 1 1 0 0 1
   mask      192 = 1 1 0 0 0 0 0 0
   AND           = 0 1 0 0 0 0 0 0 = 64
   (kept the first 2 bits, forced the other 6 to 0)
   → 156.180.150.64

LAST address = address OR NOT(mask)  → set the host bits
   NOT(192)      = 0 0 1 1 1 1 1 1  = 63
   last octet 89 = 0 1 0 1 1 0 0 1
   OR            = 0 1 1 1 1 1 1 1 = 127
   (kept the first 2 bits, forced the other 6 to 1)
   → 156.180.150.127

COUNT = NOT(mask) + 1 = 63 + 1 = 64  (= 2^(32−26))
```

ANALOGY The mask is a **stencil**. The 1s are holes you can see through; the 0s are covered. **AND** shows you only the network part with the rest blanked to 0 — that's the **first** address. **OR with the inverted stencil** fills the covered part solid with 1s — that's the **last** address.

HOOK **"AND for first, OR-NOT for last, NOT+1 for the count."** Say it three times. It is the only thing you need for Strike Sheet Q20, and it feeds Q21–Q23 too.

TRAP You almost never need to touch the first three octets — for any prefix of 24 or more they are unchanged. **Do the binary on the last octet only.** That turns a scary question into eight columns of arithmetic.

THIS UNLOCKS Strike Sheet Q20 and the method box at the top of Unit 3.

10 Why usable hosts = N − 2, and what "network" and "broadcast" addresses are

In every block of N addresses, **two are reserved and cannot be given to a device**:

- The **first** address (all host bits = 0) is the **network address**. It names the *network itself*, and it is what a router looks up in its **forwarding table** to decide which interface to send a packet out of.
- The **last** address (all host bits = 1) is the **broadcast address**. A packet sent to it goes to **every** host on that network.

$$\text{usable hosts} = N - 2 = 2^{32-n} - 2$$

So a **/26** has 64 addresses but only **62** usable; a **/28** has 16 but only **14**; a class C has 256 but only **254**.

ANALOGY In a block of flats, flat numbers run 0 to 63 — but "0" is the name of the **building itself** on the gate, and "63" is the **public address system** that reaches every flat. Neither can be a person's home. 62 families can live there.

TRAP — THIS ONE COSTS MARKS EVERY YEAR When a question says *"total number of **valid** IP addresses"*, the word is ambiguous. **Write both**: "**64** addresses in the block, of which **62** are usable host addresses." You cannot be marked down for the complete answer.

HOOK **First and last are always spoken for.** First = the network's name, last = shout to everyone.

THIS UNLOCKS Strike Sheet Q20, Q23, Q24 and the mask table.

PART C How the conversation works

Lessons 11 – 13

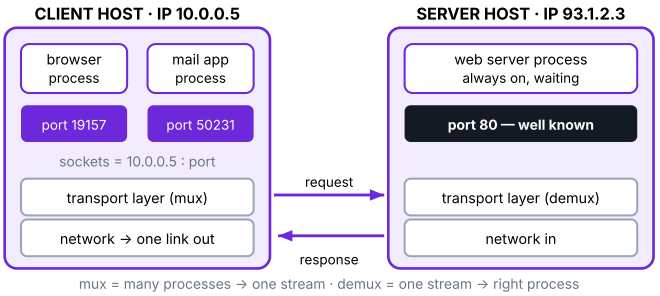
11 Processes, clients, servers and sockets

Computers don't talk to computers. Programs talk to programs.

- A **program** is the file on disk. A **process** is that program *while it is running*. Networking happens between processes.
- A **client** is the process that **starts** the conversation by sending a request.
- A **server** is the process that is **always on**, waiting, and sends a response. It has a **fixed, well-known address** so clients can find it.
- A **socket** is the doorway between a process and the network. **Socket = IP address + port number.**

This is why the transport layer's job is called **process-to-process delivery** (sometimes *service-to-service*). The network layer only manages **host-to-host** — it gets the data to the right machine and stops there.

ANALOGY The **IP address** gets the letter to the right building, the **port** to the right flat, and the **socket** is that flat's letterbox — the slot the program reads from and writes into.



Client-server vs P2P. In **client-server**, one always-on server with a fixed address serves everyone, and clients never talk to each other. In **peer-to-peer (P2P)**, the hosts talk directly and each peer both downloads *and* uploads — which is why P2P is called **self-scaling**: every new peer adds capacity instead of only adding load.

HOOK **The client knows where the server is; the server does not know where the client is** until the client speaks. That's why servers get fixed well-known ports and clients get random high ones.

THIS UNLOCKS Strike Sheet Q18 (mux/demux, 2-tuple vs 4-tuple), the Unit 1 insurance page (P2P) and every "client/server" sentence in Unit 1.

12 Bits vs bytes, Mbps, and the four delays

Units — get this wrong and a whole numerical collapses

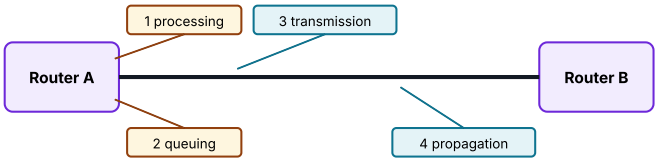
Symbol	Means	Note
b (small)	**bit**	Link speeds are in bits: Mb**ps** = megabits per second
B (capital)	**byte** = 8 bits	File and buffer sizes are in bytes: RcvBuffer = 4096 **B**
Kbps / Mbps	1,000 / 1,000,000 bits per second	**300 Kbps = 0.3 Mbps**
Gbit	1,000 Mbits	**15 Gbits = 15,000 Mbits**

TRAP In the P2P numerical the peers upload at **300 Kbps** and the server at **30 Mbps**. If you add 300 + 30 you get a wrong answer. Convert first: **0.3 Mbps**. The Strike Sheet flags this; now you know why.

```
transmission time = L / R = (packet length in bits) / (link rate in bits per second)
```

A 1 Mbit object onto a 15 Mbps link takes 1/15 s. That is **transmission** time, not travel time.

The four delays a packet experiences at each hop



total nodal delay = 1 + 2 + 3 + 4

RTT = the whole round trip, there and back

Delay	Caused by
1 Processing	Router examines the header and decides the outgoing link
2 Queuing	Waiting in the buffer behind other packets. **The only one that varies** — this is what causes **jitter** and what **congestion** makes worse.
3 Transmission	Pushing all the bits onto the link = **L / R**
4 Propagation	The signal physically travelling down the wire

Traffic intensity = (arrival rate × packet size) / link rate. As it approaches **1**, queuing delay **grows without bound** — that is the whole point of the web-caching numerical, where the access link hits intensity exactly 1.

HOOK **Transmission = how long to get the bits *out of the door*. Propagation = how long they take to *travel*.** A truck loading (transmission) is separate from the truck driving (propagation).

THIS UNLOCKS The web-caching numerical, the P2P numerical, flow-control's 100 Mbps question, and the words "jitter" and "congestion".

13 RTT — the unit everything in this course is priced in

RTT (round-trip time) = the time for a small packet to travel from client to server and back.

It includes propagation, queuing and processing delays. Since you cannot make light faster, RTT is the course's **fundamental cost unit**: protocols are judged by **how many RTTs** they need.

Opening a TCP connection costs 1 RTT (the first two parts of the handshake); **requesting an object and getting it back costs another**. Hence the formula running through all of HTTP:

```
response time per object = 2 RTT + transmit time
```

And that single formula is why persistent connections were invented: reusing one connection saves the setup RTT on every object after the first. **11 objects: 22 RTT non-persistent, 12 RTT persistent, ~3 RTT pipelined.**

Where RTT shows up again

- **TCP's timeout.** How long before a sender assumes loss? Slightly longer than one RTT — but RTT fluctuates, so TCP keeps a smoothed average (**EstimatedRTT**) plus a margin of **4 × DevRTT**. That is all Strike Sheet Q14 is.
- **Congestion control.** cwnd grows "1 MSS **per RTT**" in congestion avoidance and **doubles every RTT** in slow start. The RTT is the clock the algorithm ticks on.
- **Fairness.** Connections with a **shorter RTT grow faster** and grab more bandwidth — a known unfairness in TCP.

ANALOGY RTT is the "ping" number gamers complain about. 20 ms feels instant; 300 ms feels broken. Every protocol decision in Units 1 and 2 is really a decision about **how many times you are willing to pay that number**.

HOOK **Count RTTs, not seconds.** When any HTTP or TCP question asks "which is faster and why", the answer is always "because it costs fewer round trips."

THIS UNLOCKS Strike Sheet Q6 (persistent vs non-persistent), Q14 (RTT numerical) and Q11 (congestion control).

14 TCP vs UDP — the one split the whole of Unit 2 hangs on

The network layer (IP) is unreliable by design. It does its best and makes no promises. Everything TCP does is an attempt to build reliability on top of something that has none.

IP is **connectionless** and **best-effort**: each packet is sent independently with no guarantee — it may be lost, duplicated, corrupted or arrive out of order. The transport layer offers **two choices** about what to do with that.

	UDP	TCP
Full name	User Datagram Protocol	Transmission Control Protocol
Connection	**Connectionless** — just send	**Connection-oriented** — handshake first
Reliability	None. Lost is lost.	**Reliable** — detects loss and retransmits
Ordering	No guarantee	Delivered **in order**
Header	**8 bytes**	**20 bytes** (up to 60)
Flow control	No	Yes (**rwnd**)
Congestion control	No	Yes (**cwnd**)
State at the ends	None kept	Buffers, sequence numbers, timers
Socket is	**2-tuple** (dest IP, dest port)	**4-tuple** (both IPs + both ports)
Used by	**DNS**, voice/video calls, SNMP, NFS	HTTP, SMTP, FTP, POP3/IMAP

ANALOGY **UDP is shouting across a room** — instant, no setup, and if someone didn't catch a word, too bad, the conversation has moved on. **TCP is a phone call** — you dial, both say hello, and if the line crackles you say "sorry, repeat

Why would anyone *choose* unreliable?

Because reliability costs **time**, and for some applications a late packet is worse than a missing one. On a video call a frame arriving 2 seconds late is useless — better to skip it. That is the whole argument, and it is worth 5 marks.

The four reasons in your slides:

1. **Finer application-level control** — UDP sends immediately; TCP may delay data for setup, congestion control or retransmission.
2. **No connection establishment** — no handshake, no setup delay. **This is why DNS uses UDP:** a DNS lookup is one tiny question and one tiny answer, and paying a handshake for it would double the cost.
3. **No connection state** — no buffers or counters per client, so one server supports far more clients.
4. **Small header** — 8 bytes instead of 20.

HOOK **TCP = Takes Care of Packets. UDP = U Don't Pcare.** And note: if an app using UDP *does* need some reliability, it builds it **itself, at the application layer**.

TRAP Both TCP and UDP do **multiplexing/demultiplexing** and both have a **checksum**. Those are the two things they share. Everything else in the table is a difference.

that". The phone call is more reliable and more expensive to set up.

THIS UNLOCKS Strike Sheet Q18, Q19, and the framing of every TCP question in Unit 2.

15 Sequence numbers, ACKs, timers and windows — how reliability is actually built

Four simple ideas, combined, give you reliable delivery over an unreliable network. Every single Unit 2 topic is one of these four, examined closely.

1. Sequence number — "this is piece number X"

Number each piece so the receiver can **detect a gap**, **reorder** what arrives out of order, and **discard duplicates**.

The TCP twist that confuses everyone: TCP numbers **bytes, not segments**. A segment's sequence number is the **byte-stream number of its first data byte** — so Seq = 100 carrying 5 bytes holds bytes 100 to 104.

2. Acknowledgement (ACK) — "I got up to here"

The receiver sends back a small message confirming receipt. In TCP the ACK number is the **next byte expected**, not the last one received:

```
ACK = last byte received + 1
```

So after the segment above, the receiver replies **ACK = 105**. And TCP's ACKs are **cumulative**: ACK = 105 means "*everything* before byte 105 arrived", not just that one segment. That single property is why a lost ACK is often harmless — the next ACK covers it.

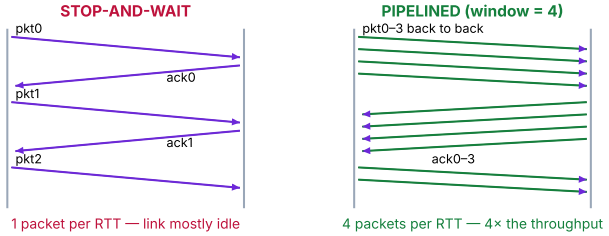
3. Timer — "it's been too long, resend"

If no ACK returns within the **TimeoutInterval**, the sender assumes loss and **retransmits**. Setting that interval is the RTT-estimation problem from Lesson 13: too short and you resend data that was merely slow; too long and you sit idle after a real loss.

4. Window — "how much may I have in flight at once?"

The naive protocol is **stop-and-wait**: send one packet, wait for its ACK, send the next. Correct, but catastrophically slow — you pay a full RTT per packet and the link sits idle throughout.

The fix is **pipelining**: allow **N unacknowledged packets in flight** at once. N is the **window size**. This one idea is behind **Go-Back-N**, **Selective Repeat**, **flow control** and **congestion control** — all four just answer *"what should N be, and what happens when something in the window is lost?"*



TESTED MOST **rwnd vs cwnd** limit the same thing — data in flight — for different reasons. **rwnd protects the receiver** from buffer overflow; **cwnd protects the network** from congestion. The sender obeys the smaller: **in flight = min(cwnd, rwnd)**.

How the four ideas map onto every Unit 2 topic

Topic in the Strike Sheet	Which of the four it is really about
Go-Back-N	Window + cumulative ACK + **one** timer. On loss, resend the whole window because the receiver kept **no buffer**.
Selective Repeat	Window + **individual** ACKs + one timer **per packet**. The receiver buffers, so only the lost packet is resent — hence the non-overlapping windows, 2^{m-1}.
TCP reliable data transfer	Byte-based sequence numbers + cumulative ACKs + one timer for the **oldest unACKed** segment. Three sender events: data from above, timeout, ACK received.
Flow control (rwnd)	The window, sized by **how much space the receiver has left**.
Congestion control (cwnd)	The same window, sized by **what the network looks able to take**. Loss is the signal; the four mechanisms are the resizing rules.
Three-way handshake	Agreeing the **starting sequence numbers** before any of the above can work. Each side announces its ISN and has it acknowledged — three messages.

HOOK **Number it, acknowledge it, time it, limit it.** Those four verbs are the whole of reliable transport — when a Unit 2 question confuses you, ask which of the four it pokes at. **Unlocks Strike Sheet Q11–Q17**, effectively the entire Unit 2 section.

16 Putting it together — one web page, from keystroke to screen

Everything in Units 1, 2 and 3 happens inside this one sequence. You type `www.someschool.edu/index.html` and press Enter — if you can narrate what follows, you understand the course.

#	What happens	Which idea it is	Where it's examined
1 2	Your browser needs an **IP address** but only has a **hostname**, so it asks the **local DNS server**, which queries **root → TLD → authoritative** iteratively and returns the IP — over **UDP, port 53**, with no handshake, because it is one tiny question and one tiny answer.	The hostname → IP problem; DNS hierarchy; why UDP (Lessons 5, 14)	**Unit 1** — Q1–Q3 · **Unit 2**
3	The browser opens a **TCP connection** to that IP on **port 80**: **SYN → SYN-ACK → ACK**. Both sides announce their initial sequence numbers and allocate buffers. **Cost: 1 RTT.**	Three-way handshake; sequence numbers (Lesson 15)	**Unit 2** — Q15
4	It sends `GET /index.html HTTP/1.1` with its header lines. If a **cookie** exists for this site the browser attaches it automatically — because HTTP itself remembers nothing.	HTTP message format; statelessness	**Unit 1** — Q4, Q7
5	Going down the stack the request is wrapped: **message → segment** (ports added) → **datagram** (IPs added) → **frame** (next-hop MAC added) → **bits**.	Encapsulation; the four addresses (Lessons 4, 5)	**Unit 1** — Q8, Q9
6	At every router the **frame is rebuilt with a new MAC** while the **IP header is untouched**. The router takes the destination IP, finds its **network address**, looks it up in the **forwarding table** and picks an interface.	Masks, network addresses, forwarding (Lessons 9, 10)	**Unit 3** — Q20–Q26
7	At the server the stack unwraps in reverse. The transport layer reads **destination port 80** and **demultiplexes** to the web-server process — matched on the full **4-tuple**, so thousands of clients on port 80 stay separate.	Demultiplexing; sockets (Lesson 11)	**Unit 2** — Q18
8	The server replies `HTTP/1.1 200 OK` plus the file. TCP cuts it into segments (data ≤ **MSS 1460 B**), numbers the **bytes**, and starts a **timer** on the oldest unacknowledged one.	Segmentation, sequence numbers, timers (Lesson 15)	**Unit 2** — Q16
9	It starts cautiously — **cwnd = 1 MSS**, doubling every RTT (**slow start**) — and never exceeds the **rwnd** advertised in each ACK. **In flight = min(cwnd, rwnd).**	The window idea, twice over (Lesson 15)	**Unit 2** — Q11, Q17
10	A segment is lost. The client re-ACKs the last in-order byte; after **3 duplicate ACKs** the server **fast-retransmits**, halves ssthresh and enters **fast recovery**. Had the *timer* expired instead, cwnd would have collapsed to 1 MSS.	Loss detection and its two different recoveries	**Unit 2** — Q11, Q12
11 12	The browser parses the HTML, finds 10 images and requests them **over the same connection** (HTTP/1.1 is persistent) rather than paying the setup RTT ten more times. Then **FIN → ACK → FIN → ACK** closes it — four segments, because TCP is full-duplex and each direction shuts separately; the client waits in **TIME_WAIT** in case its final ACK was lost.	Counting RTTs; connection teardown (Lessons 13, 15)	**Unit 1** — Q6 · **Unit 2** — Q15

REVISION TEST Cover the right-hand columns and narrate these steps out loud. Any step you stumble on points at one lesson here and one card in the Strike Sheet — step 6 hazy means re-read Part B; steps 8–10 mean re-read Lesson 15.

DECODER 1 Acronyms, expanded

every short form used in the Strike Sheet

General & Unit 1 ACKAcknowledgement ARPANETAdvanced Research Projects Agency Network — the Internet's ancestor CNAMECanonical Name (a DNS record type) CRLFCarriage Return + Line Feed (the blank line in HTTP) DARPADefense Advanced Research Projects Agency — built TCP/IP DNSDomain Name System DSLDigital Subscriber Line FTPFile Transfer Protocol HTTPHyperText Transfer Protocol HTTPSHTTP Secure IANAInternet Assigned Numbers Authority — coordinates the root servers ICANNInternet Corporation for Assigned Names and Numbers — allocates address blocks, accredits registrars IMAPInternet Mail Access Protocol ISOInternational Organization for Standardization (the **body**, est. 1947) ISPInternet Service Provider LANLocal Area Network MACMedia Access Control (the 48-bit hardware address) MXMail eXchange (a DNS record type) NICNetwork Interface Card NSName Server (a DNS record type) OSIOpen Systems Interconnection (the **7-layer model**) P2PPeer-to-Peer POP3Post Office Protocol version 3 RRResource Record (a DNS entry) SMTPSimple Mail Transfer Protocol SSLSecure Socket Layer TLDTop-Level Domain (.com, .edu, .in) TTLTime To Live (DNS caching; also the IPv4 hop limit) UAUser Agent (your mail app or browser) URLUniform Resource Locator WANWide Area Network

Unit 2 — transport AIMDAdditive Increase, Multiplicative Decrease ARQAutomatic Repeat reQuest cwndCongestion WiNDow CWRCongestion Window Reduced (TCP flag) DevRTTDeviation of RTT ECEECN Echo (TCP flag) ECNExplicit Congestion Notification EWMAExponentially Weighted Moving Average (how EstimatedRTT is smoothed) FINFinish (TCP flag) FSMFinite State Machine GBNGo-Back-N ISNInitial Sequence Number MSSMaximum Segment Size (typically 1460 B) MTUMaximum Transfer / Transmission Unit (1500 B on Ethernet) PSHPush (TCP flag) RDTReliable Data Transfer RSTReset (TCP flag) RTTRound-Trip Time rwndReceive WiNDow SRSelective Repeat ssthreshSlow-Start THRESHold SYNSynchronize (TCP flag) TCPTransmission Control Protocol UDPUser Datagram Protocol URGUrgent (TCP flag)

Unit 3 — network layer BGPBorder Gateway Protocol CIDRClassless Inter-Domain Routing DF / MFDon't Fragment / More Fragments (IPv4 flags) HLENHeader LENgth ICMPInternet Control Message Protocol IPInternet Protocol IPsecIP Security OSPFOpen Shortest Path First QoSQuality of Service RIPRouting Information Protocol VERVERsion field

Symbols you will meet	
/n	CIDR prefix length — how many of the 32 bits are the network part
2^n	Address-space size with n bits
$\log_2 N$	"2 to the power of what gives N" — your powers table, read backwards
α (alpha)	0.125 — weight of a new sample in EstimatedRTT
β (beta)	0.25 — weight of a new deviation in DevRTT
Σu_i	"sum of all the peers' upload rates" (P2P formula)
d_{min}	The slowest peer's download rate
≤ / ≥	At most / at least
m	Number of bits in a sequence number, so 2^m distinct numbers

NOTE Two pairs are worth double-checking in an answer: **ISO vs OSI** (body vs model) and **MTU vs MSS** (whole frame payload, 1500 B, vs data in one TCP segment, 1460 B).

DECODER 2 Phrases that trip people up

look these up while reading the Strike Sheet

A – D	
Best-effort	Tries to deliver, promises nothing. IP is best-effort.
Broadcast address	Last address of a block (all host bits 1); reaches every host on that network.
Connectionless	No setup before sending; each packet independent. UDP and IP.
Connection-oriented	Handshake first, state kept at both ends. TCP.
Cumulative ACK	"ACK n" means *everything* before n arrived, not just one segment.
Decapsulation	Stripping headers off on the way up the stack.
Demultiplexing	Delivering an arriving segment to the correct socket / process.
Duplicate ACK	A repeat of the same ACK: "I'm still missing that piece." Three of them trigger fast retransmit.
Encapsulation	Adding a header on the way down the stack.

E – N	
End-to-end	Between the two hosts at the ends: application, transport, network.
Full-duplex	Both directions at once. Why closing TCP takes 4 segments.
Hop-to-hop	Across one link only: data link and physical.
Host-to-host	Machine to machine (network layer), as opposed to process-to-process (transport).
In flight	Sent but not yet acknowledged.
Jitter	Variation in packet arrival time, caused by queuing delay.
Logical connection	An imaginary direct link between peer layers. Only the physical layer really moves bits.
Multiplexing	Gathering data from many processes into one outgoing stream.
Network address	First address of a block (all host bits 0); names the network, used in forwarding tables.

O – T	
Octet	8 bits — one of the four numbers in an IPv4 address.
Pipelining	Sending several requests or packets back-to-back without waiting for each reply.
Prefix / suffix	Network part / host part of an IP address.
Process-to-process	Program to program — the transport layer's job.
Self-scaling	Capacity grows as users do. P2P is self-scaling; client-server is not.
Socket	IP address + port number; a process's doorway to the network.
Stateless	The server remembers nothing between requests. HTTP is stateless; cookies work around it.
Traffic intensity	(arrival rate × size) / link rate. As it nears 1, delay explodes.
Well-known port	A port in 0–1023 reserved for a standard service (80, 25, 53 …).

THE SIX PAIRS PEOPLE MIX UP

Don't confuse	with
ISO — the standards body (1947)	**OSI** — the 7-layer model it published
Routing — choosing the path in advance	**Forwarding** — moving one received packet to its next hop
Flow control (rwnd) — protects the *receiver*	**Congestion control (cwnd)** — protects the *network*
Transmission delay — getting bits out of the door (L/R)	**Propagation delay** — bits travelling down the wire
MTU 1500 B — largest frame payload	**MSS** 1460 B — largest data in one TCP segment
internet — any joined networks	**Internet** — the global one

READING AN EXAM QUESTION'S VERB

Verb	What the marks are actually for
Illustrate / Show / Draw	A **labelled diagram** is ~2 of the marks. Draw it first.
Compare / Differentiate	A **table** with 4–6 rows, not two paragraphs.
Apply / Solve / Calculate	Write the **formula**, then substitute, then the answer with units. Working carries the marks.
Analyze / Justify	Numbered reasons, each with a consequence. Say *why*, not just *what*.
Identify / List	A numbered list with one line each. Say how many there are up front.
Discuss / Explain	Definition → structure → diagram or table → one example.

CHECK Can you answer these without looking?

if yes, open the Strike Sheet

Part A — the model

1. What is the difference between a host and a connecting device?
2. How many layers is a router involved in? A switch?
3. What is a protocol, in one sentence?
4. What does encapsulation mean?
5. Name the object at each of the five layers, top to bottom.
6. Which address changes at every hop, and which stays the same?
7. What question does a **port number** answer that an **IP address** cannot?
8. Which two OSI layers are missing from TCP/IP?
9. What is the difference between ISO and OSI?
10. Which layers are end-to-end and which are hop-to-hop?

Part B — the maths

11. What is 2^6? 2^8? 2^{10}?
12. Convert 89 to binary. Convert 11000000 to decimal.
13. What is the mask for /26 in dotted decimal?
14. What is $\log_2 128$, and what prefix does a 128-address block get?
15. 89 AND 192 = ? 89 OR 63 = ?
16. Why do you AND to get the first address and OR-NOT to get the last?
17. A /28 block: how many addresses, how many usable hosts?
18. Why is it N − 2 and not N?
19. A company needs 50 addresses. What block do they get and why not 50 exactly?
20. Why must the largest subnet be allocated first?

Part C — the conversation

21. What is a socket made of?
22. Why does a client get a random high port but a server a fixed low one?
23. 300 Kbps in Mbps?
24. Which of the four delays varies, and what does that cause?
25. What is RTT, and why is a non-persistent HTTP object 2 RTT?
26. Give two differences between TCP and UDP, and one thing they share.
27. Why does DNS use UDP?
28. In TCP, what does an ACK number actually mean?
29. What is a window, and why is stop-and-wait slow?
30. rwnd vs cwnd — what does each protect?

If you got most of those — you are ready

Open **DCN_CIE1_Strike_Sheet.pdf** and start at its page 2 (the pattern decoder). Nothing in those 34 pages will be unfamiliar vocabulary any more; what remains is the **specific content** of 27 questions that have genuinely been asked before.

If three or more stumped you, re-read just the lessons they belong to — each question above is numbered in lesson order, so question 15 is Lesson 9, question 25 is Lesson 13, and so on. Then go.

Ground Zero · companion to the CS54 CIE-1 Strike Sheet · Units 1, 2 and 3 up to IPv4 addressing